Практическая работа 6. МЕТОДОЛОГИЯ MSF

В 1990-х годах компания Microsoft, стремясь достичь максимальной отдачи от реализации заказных IT-решений и в целях улучшения работы с субподрядчиками, обобщила свой опыт по разработке, внедрению, сопровождению и консалтингу ПО, создав методологию MSF. В 2002 г. вышла версия MSF 3.1, состоявшая из пяти документов-руководств:

- модели процессов,
- модели команды,
- модели управления проектами,
- дисциплины управления рисками,
- управления подготовкой.

IT-решение понимается как скоординированная поставка набора элементов (таких, как: программные средства, документация, обучение и сопровождение), необходимых для удовлетворения бизнес-потребности конкретного заказчика. Причем под его разработкой понималось создание ПО, обучение пользователей и полная передача продукта команде сопровождения. Задача наладки полноценного сопровождения IT-решения – важная составляющая успешности проекта.

Основными новшествами MSF являются следующие:

1. Акцент на внедрении IT-решения.
2. Модель процесса, объединяющая спиральную и водопадную модели.
3. Особая организация команды – не иерархическая, а как группа равных, но выполняющих разные функции (роли) работников.
4. Техника управления компромиссами.

Ниже мы рассмотрим эти положения более детально.

В 2005 г. MSF претерпела значительные изменения. Версия MSF 4.0. стала составной частью продукта Visual Studio Team System (VSTS) и разделилась на две ветки – MSF for Agile и MSF for CMMI. При этом, если версии до 3.x были именно методологиями (там были изложены принципы, MSF свободно распространялась в виде Word-документов, которые были также

переведены на русский язык), то теперь MSF превратилась в шаблоны процесса для VSTS. Эти шаблоны имеют описание в виде html-документов (Word-документов уже нет) и определяют типы ролей, их ответственность, действия в рамках этой ответственности, а также все входные и выходные артефакты этих деятельностей и другие формализованные атрибуты процесса разработки. Кроме этого «человеческого» описания MSF for Agile и MSF for CMMI имеют XML-настройки, которые позволяют в точности следовать предложенным выше описаниям, используя VSTS. При этом на процесс накладываются достаточно жесткие ограничения, деятельность разработчиков сопровождается набором автоматических действий – все это задано в шаблонах. Данные шаблоны можно частично использовать (например, без некоторых ролей), а также изменять (VSTS представляет обширные средства настойки шаблонов). Версия MSF 4.2 продолжила направление версии MSF 4.0.

Можно считать, что фактически версии MSF 4.x являются продуктами другого класса, чем MSF 3.x. MSF 3.x были нацелены на разработку заказных IT-решений, MSF 4.0 – на разработку произвольного ПО. Формально документация этих версий несильно пересекается и содержит для 3.x в большей степени общие принципы, а для 4.x – формальные атрибуты в терминах VSTS. В некотором смысле можно сказать, что MSF 4.x является реализацией MSF 3.x для продукта VSTS.

Основные принципы

Перечислим основные принципы MSF.

1. *Единое видение проекта.* Успех коллективной работы над проектом немыслим без наличия у членов проектной группы и заказчика единого видения (shared vision), т.е. четкого и, самое главное, одинакового понимания целей и задач проекта. Как проектная группа, так и заказчик изначально имеют собственные предположения о том, что должно быть достигнуто в ходе работы над проектом. Лишь наличие единого видения способно внести ясность и обеспечить движение всех заинтересованных в проекте сторон к общей цели. Формирование единого видения и последующее следование

ему являются столь важными, что модель процессов MSF выделяет для этой цели специальную фазу – «Выработка концепции», которая заканчивается соответствующей вехой.

2. *Гибкость – готовность к переменам.* Традиционная дисциплина управления проектами и каскадная модель исходят из того, что все требования могут быть четко сформулированы в начале работы над проектом, и далее они не будут существенно изменяться. В противоположность этому MSF основывается на принципе непрерывной изменяемости условий проекта при неизменной эффективности управленческой деятельности.
3. *Концентрация на бизнес-приоритетах.* Независимо от того, нацелен ли разрабатываемый продукт на организации или индивидуумов, он должен удовлетворить определенные нужды потребителей и принести в некоторой форме выгоду или отдачу. В отношении индивидуумов это может означать, например, эмоциональное удовлетворение – как в случае компьютерных игр. Что же касается организаций, то неизменным целевым фактором продукта является бизнес-отдача (business value). Обычно продукт не может приносить отдачу до того, как он полностью внедрен. Поэтому модель процессов MSF включает в свой жизненный цикл не только разработку продукта, но и его внедрение.
4. *Поощрение свободного общения.* Исторически многие организации строили свою деятельность на основе сведения информированности сотрудников к минимуму, необходимому для исполнения работы (need-to-know). Зачастую такой подход приводит к недоразумениям и снижает шансы команды на достижение успеха. Модель процессов MSF предполагает открытый и честный обмен информацией как внутри команды, так и с ключевыми заинтересованными лицами. Свободный обмен информацией не только сокращает риск возникновения недоразумений, недопонимания и неоправданных затрат, но и обеспечивает максимальный вклад всех участников проектной группы в снижение существующей в проекте неопределенности. По этой причине модель процессов MSF предлагает проведение анализа хода работы над

проектом в определенных временных точках. Документирование результатов делает ясным прогресс, достигнутый в работе над проектом - как для проектной команды, так и для заказчика и других заинтересованных в проекте сторон.

Модель жизненного цикла

Модель жизненного цикла сочетает в себе свойства двух стандартных моделей: каскадной и спиральной (рис. 1).

В основе методологии MSF лежит *итеративный интегрированный подход* к созданию и внедрению решений, базирующийся на *фазах* и *вехах*.

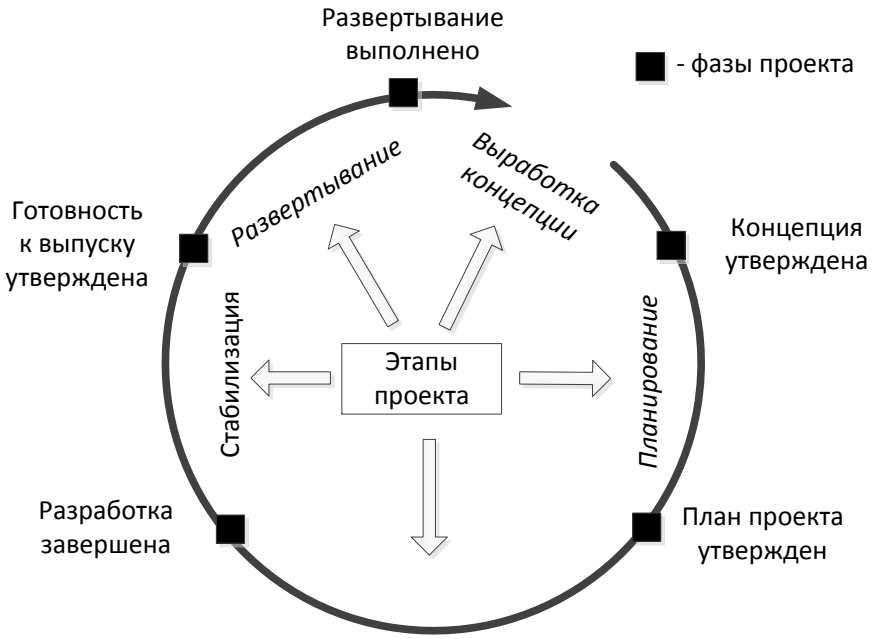


Рисунок. 1 – Модель жизненного цикла MSF

Итеративность подхода предусматривает поэтапное созда-ние всех элементов проекта: программного кода, документации, дизайна, планов. Реализацию проекта фрекомендуется начинать с построения, тестирования и внедрения базовой функционально-сти системы. Затем к решению добавляются все новые и новые возможности. Такой подход к процессу разработки подразумева-ет достаточную гибкость в ведении документации. Проектные документы должны изменяться по мере эволюции проекта. Их пересмотр не прекращается до конца проекта и производится по-

сле каждой итерации. Такой подход существенно отличается от принципов ведения документации в каскадной модели, где процесс разработки начинается лишь после того, как готовы и зафиксированы все требования и спецификации.

Интеграция в рамках одного проекта процедур разработки и внедрения системы позволяет более полно сосредоточиться на нуждах Заказчика (даже если разработка решения прошла удачно, заказчики не увидят отдачи до тех пор, пока система не запущена в эксплуатацию), и улучшить взаимодействие с командой сопровождения.

Фазы проекта определяют последовательно решаемые задачи, а вехи – ключевые точки проекта, характеризующие достижение какого-либо существенного результата.

В MSF используются два вида вех: главные и промежуточные. Они имеют следующие характеристики:

- *главные вехи* служат точками перехода от одной фазы к другой и определяют изменения в текущих задачах ролевых кластеров проектной команды; в MSF главные вехи являются в достаточной степени универсальными для применения в любом ИТ проекте;
- *промежуточные вехи* показывают достижение определенного прогресса в исполнении фазы проекта и расчленяют большие сегменты работы на меньшие, обозримые и управляемые участки; промежуточные вехи могут варьироваться в зависимости от характера проекта.

Фаза выработки концепции

Цель фазы – создание и сплочение проектной группы на основе выработки единого видения проекта.

Основные выполняемые задачи:

- создание ядра проектной группы;
- подготовка документа общего описания (Видение) и рамок проекта (vision / scope document). Видение (vision) – это ничем не ограничиваемое представление о том, каким должно быть решение. Рамки (scope) – определение того, что из предложенного этим видением будет реализовано в условиях существующих проектных ограничений;

– определение и оценка главных рисков проекта;
– выявление и первичный анализ бизнес-требований (детально эти требования рассматриваются во время фазы планирования).

Результаты выполнения фазы фиксируются в ряде документов:

– общее описание и рамки проекта;
– документ оценки рисков;
– описание структуры проекта.

Фаза планирования

Цель фазы – разработка планов проекта.

Основные выполняемые задачи:

1. Подготовка функциональной спецификации на систему включает в себя анализ и документирование проектных требований (выделяются: бизнес-требования, потребительские требования, эксплуатационные требования и системные требования, относящиеся к решению в целом). Задача предусматривает последовательное выполнение следующих работ:

– выявления типов пользователей системы;
– выявления сценариев использования, в которых моделируется выполнение какой-либо операции определенным типом пользователя;
– выделения последовательностей специфических действий, называемых примерами пользования (use cases), которые необходимо выполнить пользователю для осуществления операции;
– проектирования (дизайн системы). В MSF выделяются три уровня процесса проектирования: концептуальный дизайн (conceptual design), логический дизайн (logical design) и физический дизайн (physical design).

Концептуальный дизайн – описание всего, что нужно включить в конечный продукт. В это описание не входит информация о способе реализации решения. Концептуальный дизайн включает только подробные сведения о функциональности предлагаемого решения, взаимодействии с существующей технологической инфраструктурой, о пользовательском интерфейсе и предполагаемых рабочих характеристиках системы.

Логический дизайн – описание состава, организации и взаимодействия элементов, из которых состоит программное решение.

Физический дизайн – описание программного решения в терминах разработчика системы. Включает все необходимые детали для реализации: технологии, организацию, структуру и взаимосвязи элементов, которые будут использованы при создании программного решения.

Результаты процесса проектирования документируются в функциональной спецификации.

2. Подготовка рабочих планов.

На основе разработанных спецификаций каждый из руководителей ролевых кластеров проектной группы подготавливает планы, относящиеся к его роли (план внедрения, план тестирования, план эксплуатации, план мер безопасности, план обучения и пр.), и принимает участие в командных сессиях планирования, где все планы синхронизируются и представляются вместе в виде сводного плана проекта.

3. Оценка проектных затрат и сроков разработки различных составляющих проекта.

Результаты фазы оформляются в *базовой версии проекта* путем создания следующих документов:

- функциональной спецификации;
- плана управления рисками;
- сводного плана и сводного календарного графика проекта.

Фаза разработки

Цель фазы – создание компонент-решения (включая как документацию, так и программный код).

Результаты фазы:

- исходный и исполнимый код приложений;
- скрипты установки и конфигурирования;
- окончательная функциональная спецификация;
- материалы поддержки решения;
- спецификации и сценарии тестов.

Фаза стабилизации

Цель фазы – тестирование и отладка разработанного решения в реалистичной модели производственной среды.

Основные выполняемые задачи:

- выявление, приоритезация и устранение ошибок;
- пилотное внедрение решения.

Результаты:

- окончательный продукт;
- документация выпуска;
- материалы поддержки решения;
- результаты и инструментарий тестирования;
- исходный и исполнимый код приложений;
- проектная документация.

Фаза внедрения

Цель фазы – установка и отладка системы в реальных условиях эксплуатации, передача системы персоналу поддержки и сопровождения, получение окончательного одобрения результатов проекта со стороны Заказчика.

Результатами этой фазы являются:

- информационные системы эксплуатации и поддержки;
- работающие процедуры и процессы;
- базы знаний, отчеты, журналы протоколов;
- версии проектных документов, массивы данных и программный код, разработанные во время проекта;
- отчет о завершении проекта;
- окончательные версии всех проектных документов;
- показатели удовлетворенности Заказчика и потребителей.

Модель команды

Основные принципы. Главной особенностью модели команды в MSF является то, что она «плоская», то есть не имеет официального лидера. Все отвечают за проект в равной степени, уровень заинтересованности каждого в результате очень высок, а коммуникации внутри группы четкие, ясные, дружественные и ответственные. Конечно, далеко не каждая команда способна так работать – собственно, начальники для того и нужны, чтобы нести основной груз ответственности за проект и во многом освободить от него других. Демократия в команде возможна при высоком уровне осознанности и заинтересованности каждого, а

также в ситуации равности в профессиональном уровне (пусть и в разных областях – см. различные ролевые кластеры в команде, о которых речь пойдет ниже). С другой стороны, в реальном проекте, в рамках данной модели команды, можно варьировать степень ответственности, в том числе вплоть до выделения, при необходимости, лидера.

Одной из особенностей отношений внутри команды является высокая культура дисциплины обязательств:

- готовность работников принимать на себя обязательства перед другими;
- четкое определение тех обязательств, которые они на себя берут;
- стремление прилагать должные усилия к выполнению своих обязательств;
- готовность честно и незамедлительно информировать об угрозах выполнению своих обязательств.

Ролевые кластеры. MSF основан на постулате о семи качественных целях, достижение которых определяет успешность проекта. Эти цели обусловливают модель проектной группы и образуют *ролевые кластеры* (или просто *роли*) в проекте. В каждом ролевом кластере может присутствовать по одному или несколько специалистов, некоторые роли можно соединять одному участнику проекта. Каждый ролевой кластер представляет уникальную точку зрения на проект, и в то же время никто из членов проектной группы в одиночку не в состоянии успешно представлять все возможные взгляды, отражающие качественно различные цели. Для разрешения этой дилеммы команда соратников (команда равных), работающая над проектом, должна иметь четкую форму отчетности перед заинтересованными сторонами при распределенной ответственности за достижение общего успеха. В MSF следующие ролевые кластеры.

- *Управление продуктом*. Основная задача этого ролевого кластера – обеспечить, чтобы заказчик остался довольным в результате выполнения проекта. Этот ролевой кластер действует по отношению к проектной группе как представитель заказчика и зачастую формируется из сотрудников органи-

зации-заказчика. Он представляет бизнес-сторону проекта и обеспечивает его согласованность со стратегическими целями заказчика. В него же входит контроль за полным пониманием интересов бизнеса при принятии ключевых проектных решений.

- *Управление программой* обеспечивает управленческие функции – отслеживание планов и их выполнение, ответственность за бюджет, ресурсы проекта, разрешение проблем и трудностей процесса, создание условий, при которых команда может работать эффективно, испытывая минимум бюрократических преград.
- *Разработка*. Этот ролевой кластер занимается собственно программированием ПО.
- *Тестирование* – отвечает за тестирование ПО.
- *Удовлетворение потребителя*. Дизайн удобного пользовательского интерфейса и обеспечение удобства эксплуатации ПО (эргономики), обучение пользователей работе с ПО, создание пользовательской документации.
- *Управление выпуском*. Непосредственно ответственен за беспрепятственное внедрение проекта и его функционирование, берет на себя связь между разработкой решения, его внедрением и последующим сопровождением, обеспечивая информированность членов проектной группы о последствиях их решений.
- *Архитектура*[1]. Организация и выполнение высокоуровневого проектирования решения, создание функциональной спецификации ПО и управление этой спецификацией в процессе разработки, определение рамок проекта и ключевых компромиссных решений.

Масштабирование команды MSF. Наличие 7 ролевых кластеров не означает, что команда должна состоять строго из 7 человек. Один сотрудник может объединять несколько ролей. При этом некоторые роли нельзя объединять. В таблице 1 ниже представлены рекомендации MSF относительно совмещения ро-

[1] Этот ролевой кластер появился в версиях MSF 4.x. До этого данная ответственность входила в ролевой кластер «Управление программой».

лей в рамках одним членом команды. «+» означает, что совмещение возможно, «+–» –что совмещение возможно, но нежелательно, «–» означает, что совмещение не рекомендуется.

В частности, нельзя совмещать разработку и тестирование, поскольку, как обсуждалось выше, необходимо, чтобы у тестировщиков был сформирован свой, независимый взгляд на систему, базирующийся на изучении требований.

Таблица 1 *– Рекомендации MSF по совмещению ролей в команде проекта*

	Управление продуктом	Управление программой	Разработка	Тестирование	Удовлетворение потребителя	Управление выпуском	Архитектура
Управление продуктом		–	–	+	+	+–	–
Управление программой	–		–	+–	+–	+	+
Разработка	–	–		–	–	–	+
Тестирование	+	+–	–		+	+	+–
Удовлетворение потребителя	+	+–	–	+		+–	+–
Управление выпуском	+ –	+	–	+	+–		+
Архитектура	–	+	+	+–	+–	+	

Модель проектной группы MSF предлагает разбиение больших команд (более 10 человек) на малые многопрофильные ***группы направлений***. Эти малые коллективы работают параллельно, регулярно синхронизируя свои усилия, каждая из групп устроена на основе модели кластеров. Это компактные мини-команды, образующие матричную организационную структуру. В них входят по одному или по нескольку членов из разных ролевых кластеров. Такие команды имеют четко определенную задачу и ответственны за все относящиеся к ней вопросы, начиная от проектирования и составления календарного графика. Например, может быть сформирована специальная группа проектирования и разработки сервисов печати.

Кроме того, когда ролевому кластеру требуется много ресурсов, формируются так называемые ***функциональные группы,***

которые затем объединяются в ролевые кластеры. Они создаются в больших проектах, когда необходимо сгруппировать работников внутри ролевых кластеров по их областям компетенции. Например, в Майкрософт группа управления продуктом обычно включает специалистов по планированию продукта и специалистов по маркетингу. Как первая, так и вторая сферы деятельности относятся к управлению продуктом: одна из них сосредоточивается на выявлении качеств продукта, действительно интересующих заказчика, а вторая – на информировании потенциальных потребителей о преимуществах продукта.

Аналогично, в команде разработчиков возможна группировка сотрудников в соответствии с назначением разрабатываемых ими модулей: интерфейс пользователя, бизнес-логика или объекты данных. Часто программистов разделяют на разработчиков библиотек и разработчиков решения. Программисты библиотек обычно используют низкоуровневый язык C и создают повторно используемые компоненты, которые могут пригодиться всему предприятию. Создатели же решения обычно соединяют эти компоненты и работают с языками более высокого уровня, такими, как, например, Microsoft Visual Basic.

Часто функциональные группы имеют внутреннюю иерархическую структуру. Например, менеджеры программы могут быть подотчетны ведущим менеджерам программы, которые, в свою очередь отчитываются перед главным менеджером программы. Подобные структуры могут также появляться внутри областей компетенций. Но важно помнить, что эти иерархии не должны затенять модель команды MSF на уровне проекта в целом.

Ключевые термины

IT решение – скоординированная поставка набора элементов, необходимых для удовлетворения бизнес-потребности конкретного заказчика.

Итеративный подход при разработке ПО – предусматривает поэтапное создание всех элементов проекта.

Фазы проекта – определяют последовательно решаемые задачи.

Вехи проекта – ключевые точки проекта, характеризующие достижение какого-либо существенного результата.

Главные вехи – служат точками перехода от одной фазы проекта к другой.

Промежуточные вехи – показывают достижение определенного прогресса в исполнении фазы проекта.

Ролевые кластеры в MSF – представляет уникальную точку зрения на проект, определяющую необходимые роли для выполнения поставленных задач на определенной фазе проекта.

Краткие итоги

При разработке программных продуктов компания Microsoft использует понятие IT – решение, под которым понимается как скоординированная поставка набора элементов (таких, как: программные средства, документация, обучение и сопровождение), необходимых для удовлетворения бизнес-потребности конкретного заказчика. Методология компании Microsoft MSF является основой систем управления жизненным циклом приложений, реализованных в виде программного продукта Visual Studio Foundation Server. В MSF имеются шаблоны для формальных (MSF for CMMI) и гибких (MSF for Agile) методологи проектирования ПО. MSF использует модель жизненного цикла, которая сочетает в себе свойства двух стандартных моделей: каскадной и спиральной. В модели MSF определены следующие фазы: выработки концепции, планирования, разработки, стабилизации и внедрения. Модель команд MSF базируется на следующих ролевых кластерах: управлении продуктом, управлении программой, разработке, тестировании, удовлетворении потребителя, управлении выпуском и архитектуре. Модель команд MSF может масштабироваться для различных по сложности и объему проектов создания ПО.

Вопросы для самопроверки

1. История развития методологии MSF.
2. Поясните содержание понятия ИТ-решение в методологии Microsoft.
3. Основные принципы MSF.
4. Поясните принцип MSF «*Единое видение проекта*».
5. Поясните принцип MSF «*Гибкость – готовность к переменам*».
6. Поясните принцип MSF «*Концентрация на бизнес-приоритетах*».
7. Поясните принцип MSF «*Поощрение свободного общения*».
8. Поясните основные принципы модели команд MSF.

9. Поясните особенности отношений внутри команды MSF.
10. Ролевые кластеры.
11. Назначение кластера *Управление продуктом.*
12. Назначение кластера *Управление программой.*
13. Назначение кластера *Разработка и Тестирование.*
14. Назначение кластера *Удовлетворение потребителя.*
15. Назначение кластера *Управление выпуском.*
16. Назначение кластера *Архитектура.*
17. Масштабирование команды MSF.
18. Модель процесса в MSF.